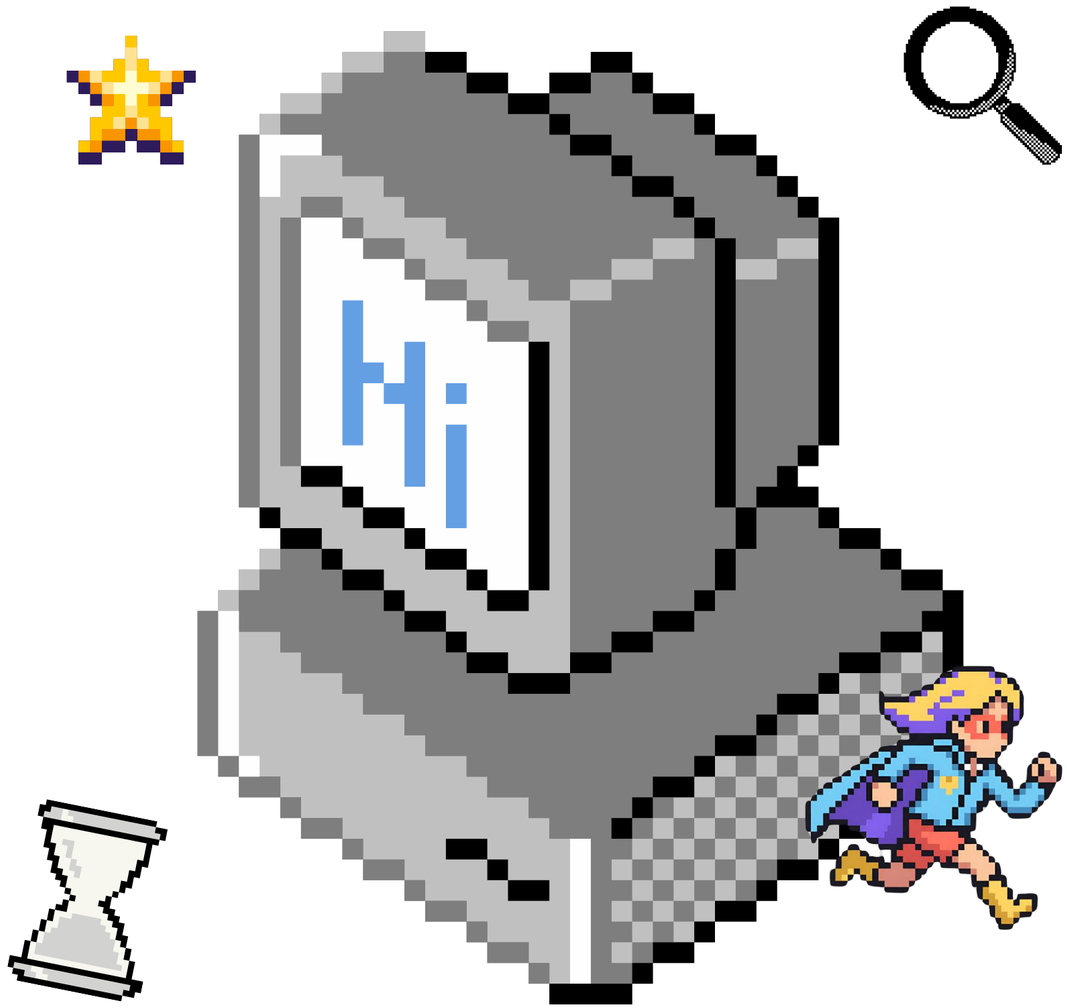


JUSTICE SQUAD

Superhero Battle - Final Project Software Design Pattern

苏睿涵 - L25020028

PROJECT OVERVIEW

A text-based Java console game

Demonstrates 8 Gang of Four Design Patterns

All patterns integrated into a single gameplay system

Core Gameplay Loop

Recruit Hero → Equip Gear → Receive City Alert → Battle Villains → Undo Actions → Track Results

TECHNOLOGY STACK



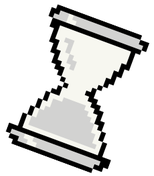
Plain Java, no external libraries, input via Scanner



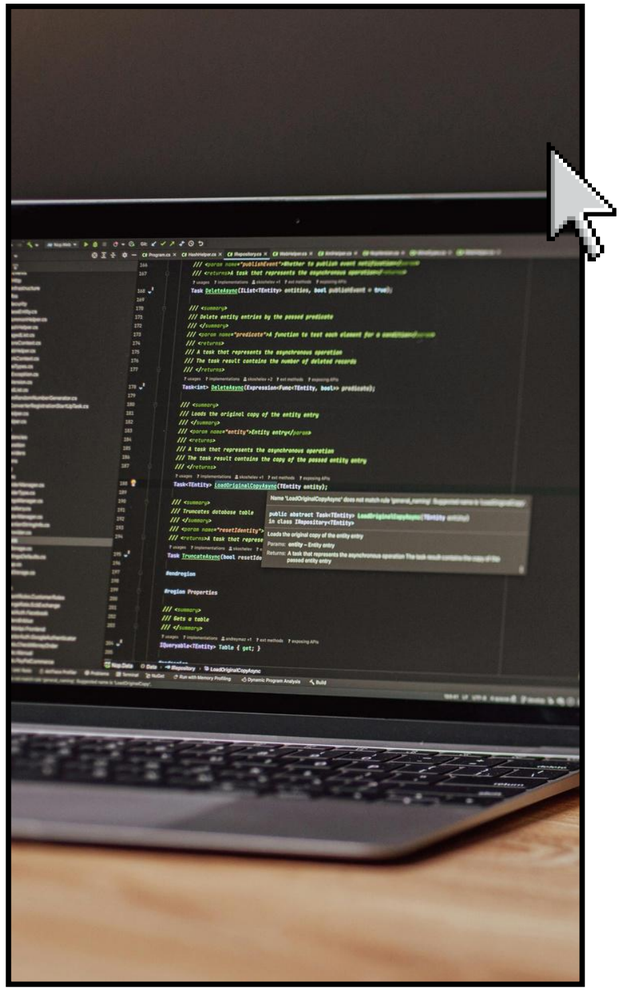
Game.java = interactive game



Main.java = scripted demo per pattern

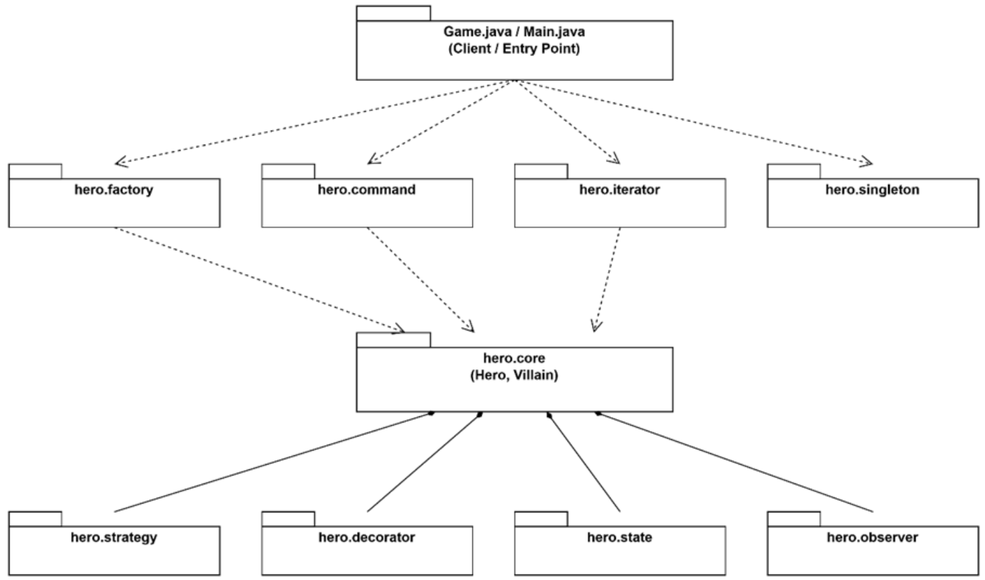






SYSTEM ARCHITECTURE

Hero is the single class where 4 patterns meet at once - that's why attack() reads like a short pipeline instead of a wall of if/else.



Entry Point

Game.java / Main.java are the only two clients of the whole codebase

Shared core

hero.core (Hero, Villain) sits in the middle - every pattern eventually touches it

Direct dependencies

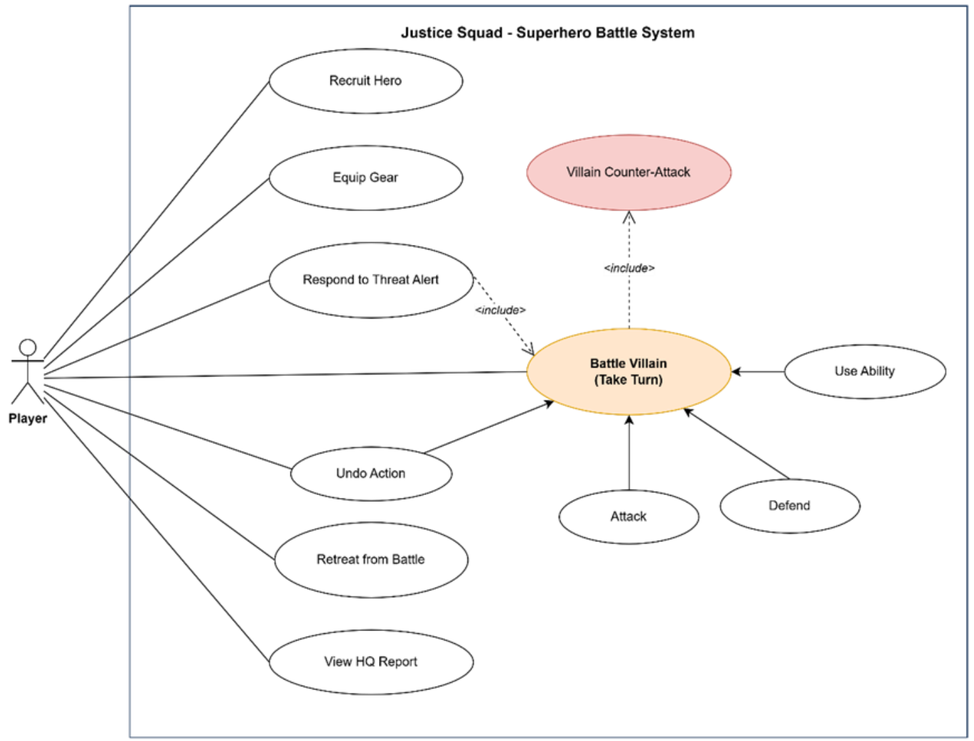
Client calls Factory, Command, Iterator, and Singleton straight away

Composed inside Hero

Strategy, Decorator, State, and Observer aren't called by the client at all - they live inside Hero itself

USECASE ANALYSIS

No second human role, no hidden menu - every action the Player doesn't choose (entering battle, the counter-attack) is wired in through an <<include>> arrow instead.



One actor, one flow

the Player drives everything - recruit a hero, equip gear, respond to a threat, then battle

Battle Villain is the core

it's the parent use case generalized by Attack, Defend, Use Ability, and Undo Action

<<include>> = automatic, not chosen

responding to a threat alert automatically includes entering battle, and Battle Villain automatically includes the villain's counter-attack



WHY THESE 8 PATTERNS?

Each pattern earns its place because it solves a problem this exact game already has - none of them were bolted on just to hit the "minimum 8" requirement.

Building the Hero

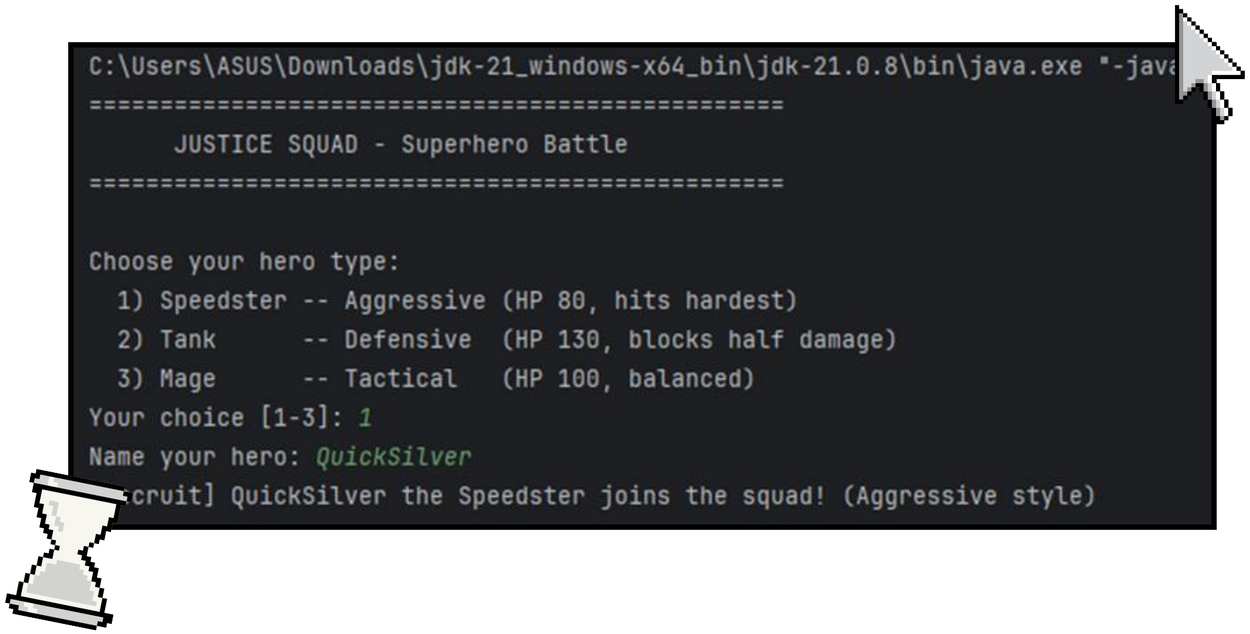
- **Factory** Method picks the right archetype
- **Decorator** stacks gear bonuses on top

Driving the battle

- **Strategy** computes damage
- **State** tracks HP-driven condition
- **Command** makes every action undoable
- **Iterator** walks the villain wave

Coordinating the session

- **Observer** broadcasts one threat to many independent reactions
- **Singleton** keeps one shared scoreboard across everything



FACTORY METHOD

Problem

Three hero archetypes (Speedster, Tank, Mage) need different HP, power, and combat style at recruitment - without an if/switch chain inside Game.java

Solution

HeroFactory.recruit() is a template method - it calls the abstract createHero(), and each concrete factory overrides it to decide which Hero gets built.

```
// Factory method - subclass decides which Hero (and style) to create.
protected abstract Hero createHero(String name);  3 implementations
```

```
// Concrete creator: a fast, fragile Speedster with Aggressive style.
public class SpeedsterFactory extends HeroFactory {  4 usages
    protected Hero createHero(String name) {
        return new Hero(name, hp: 80, basePower: 12, new AggressiveStyle());
    }
```

Without Factory Method: recruit() needs a switch on hero type.

With it: each factory subclass just knows how to build its own hero - recruit() never changes.

STRATEGY METHOD

```
  QuickSilver attacks Mr Beast for 32 damage! (Mr Beast HP: 28)
  Mr Beast strikes QuickSilver for 14!

QuickSilver  HP 66/80  [Aggressive style, gear: Suit + Cape + Shield + Gadget, state: OnMission]
Mr Beast  HP 28
State: In the field, trading blows with a villain.
```



Without Strategy: Hero needs if/else per style.

With Strategy: swap the CombatStyle object - Hero's code never changes.



Problem

Combat feel must differ per hero (hits hard vs. blocks damage) without rewriting Hero for every style.



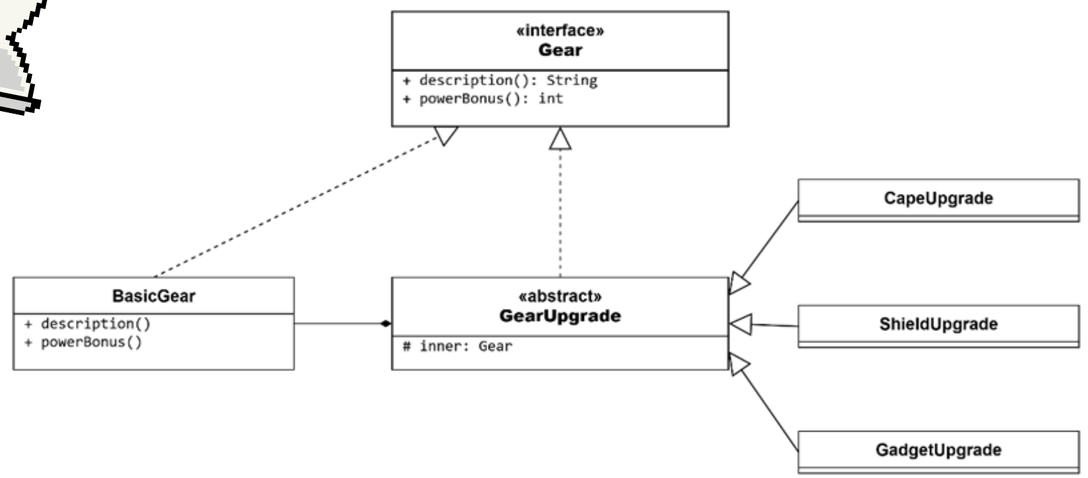
Solution

Hero never computes damage itself - it holds one CombatStyle and delegates all the math to it inside attack()

```
// Turn the hero's raw power into outgoing damage.
int attackDamage(int rawPower);  3 implementations
```

```
public int attackDamage(int rawPower) {
    return rawPower + 8;
}
```

```
Choose your gear:
  1) Suit only          (+0 power)
  2) Suit + Cape        (+3 power)
  3) Suit + Shield      (+5 power)
  4) Suit + Cape + Shield + Gadget (+12 power)
Your choice [1-4]: 2
Gear equipped: Suit + Cape (+3 power)
```



Without Decorator: 1 subclass per combination (8 combos = 8 classes).

With Decorator: 3 small classes, stacked freely, give all 8 combinations.

DECORATOR METHOD

Problem

Gear bonuses (Cape, Shield, Gadget) must stack in any combination without a subclass per combination.

Solution

GearUpgrade is both a Gear AND a holder of one - it wraps an inner layer and adds its own delta on top of whatever that layer already returns.

```
public abstract class GearUpgrade implements Gear {
    protected final Gear inner;
```

```
public int powerBonus() {
    return inner.powerBonus() + 3;
}
```

```
[ALERT] Threat at Downtown! Danger level 7. Dispatching...
QuickSilver answers the call -- heading to Downtown!
[News-CNN] BREAKING: villain spotted at Downtown (danger 7/10)!

>> QuickSilver is now OnMission
[Iterator] next villain in the wave: Mr Beast
================================================
A villain appears: Mr Beast (HP 60)
================================================
```

OBSERVER METHOD

Problem

One city threat must reach the hero AND the news, without AlertSystem knowing either of them.

Solution

AlertSystem only knows it's holding a list of AlertObserver - it has no idea one of them is a Hero and another is a NewsReporter.

Without Observer: AlertSystem must know every observer's type to notify it.

With Observer: adding a new reaction needs zero changes to AlertSystem.

```
void onThreat(String location, int dangerLevel);  2 implementations
```

```
for (AlertObserver o : observers) {
    o.onThreat(location, dangerLevel);
}
```



Without State: HP checks scattered everywhere as if/else.

With State: each condition is its own class - the hero's behavior changes automatically.

STATE METHOD

Problem

Hero condition (Available/OnMission/Injured/Retired) must change automatically with HP, not via scattered if-checks.

Solution

HeroContext holds the current HeroState and recalculates it every time HP changes - Game.java never assigns a state directly.

```
// Damage dealt is scaled by this percent (Injured heroes hit weaker).
int powerPercent();  4 implementations
```

```
// Pick the right state from current HP. Transitions happen here.
public void autoUpdate() {
    int pct = hero.getHp() * 100 / hero.getMaxHp();
    if (hero.getHp() <= 0) {
        setState(new RetiredState());
    } else if (pct < 30) {
        setState(new InjuredState());
```

```
State: In the field, trading blows with a villain.

Actions:
  1) Attack  (standard hit)
  2) Defend  (recover HP, skip attack)
  3) Ability (signature move, extra damage)
  4) Undo    (free -- reverses last action)
  0) Retreat (quit)
Your choice: 4

  Undoing: Attack (Aggressive)
  [Undo] Attack cancelled -- Mr Beast HP restored to 60.
```

COMMAND METHOD

Problem

Every battle action must be undoable, without each action knowing about every other action.

Solution

CommandHistory only ever calls execute() and undo() - it has no idea whether it's holding an attack, a defend, or a signature ability.

Without Command: undo logic gets tangled into every action type separately.

With Command: CommandHistory reverses anything through one generic interface.

```
public interface Command {
    void execute();  3 implementations
    void undo();  3 implementations
    String description();  3 implementations
}
```

```
public void undo() {
    target.takeDamage( amount: target.getHp() - villainHpBefore); // negative = heal back
    System.out.printf("  [Undo] Attack cancelled -- %s HP restored to %d.%n",
        target.getName(), target.getHp());
}
```

```
QuickSilver unleashes a SIGNATURE ABILITY on Mr B

Mr Beast is DEFEATED!
[Iterator] next villain in the wave: Dr. Octopus
================================================
A villain appears: Dr. Octopus (HP 90)
================================================
```

Without Iterator: Game.java needs to know it's a List and loop by index.

With Iterator: the storage is hidden completely - it could change tomorrow and nothing else would notice.

ITERATOR METHOD

Problem

The villain wave must be walked one at a time, without Game.java knowing how it's stored.

Solution

VillainWave implements Iterable - Game.java just writes a plain for-each loop, never touching the underlying List directly.

```
@Override
public Iterator<Villain> iterator() {
    return new WaveIterator(villains);
}
```

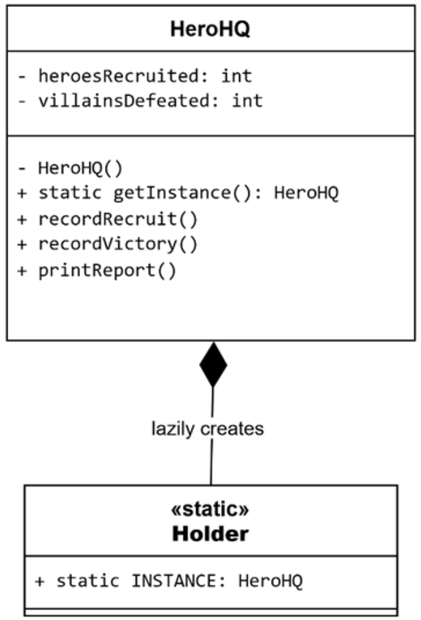
```
@Override
public Villain next() {
    if (!hasNext()) throw new NoSuchElementException("No more villains in this wave.");
    Villain villain = villains.get(index++);
    System.out.printf("  [Iterator] next villain in the wave: %s%n", villain.getName());
    return villain;
}
```

SINGLETON METHOD

```
==================================================
  All villains defeated! The city is safe.
==================================================
[HeroHQ] Report -- heroes recruited: 1 | villains defeated: 2
```

Without Singleton: counters get scattered or passed around manually.

With Singleton: 3 calls from 3 different places, still the exact same object.



Problem

Recruit/victory counters must be shared from one place across the whole session.

Solution

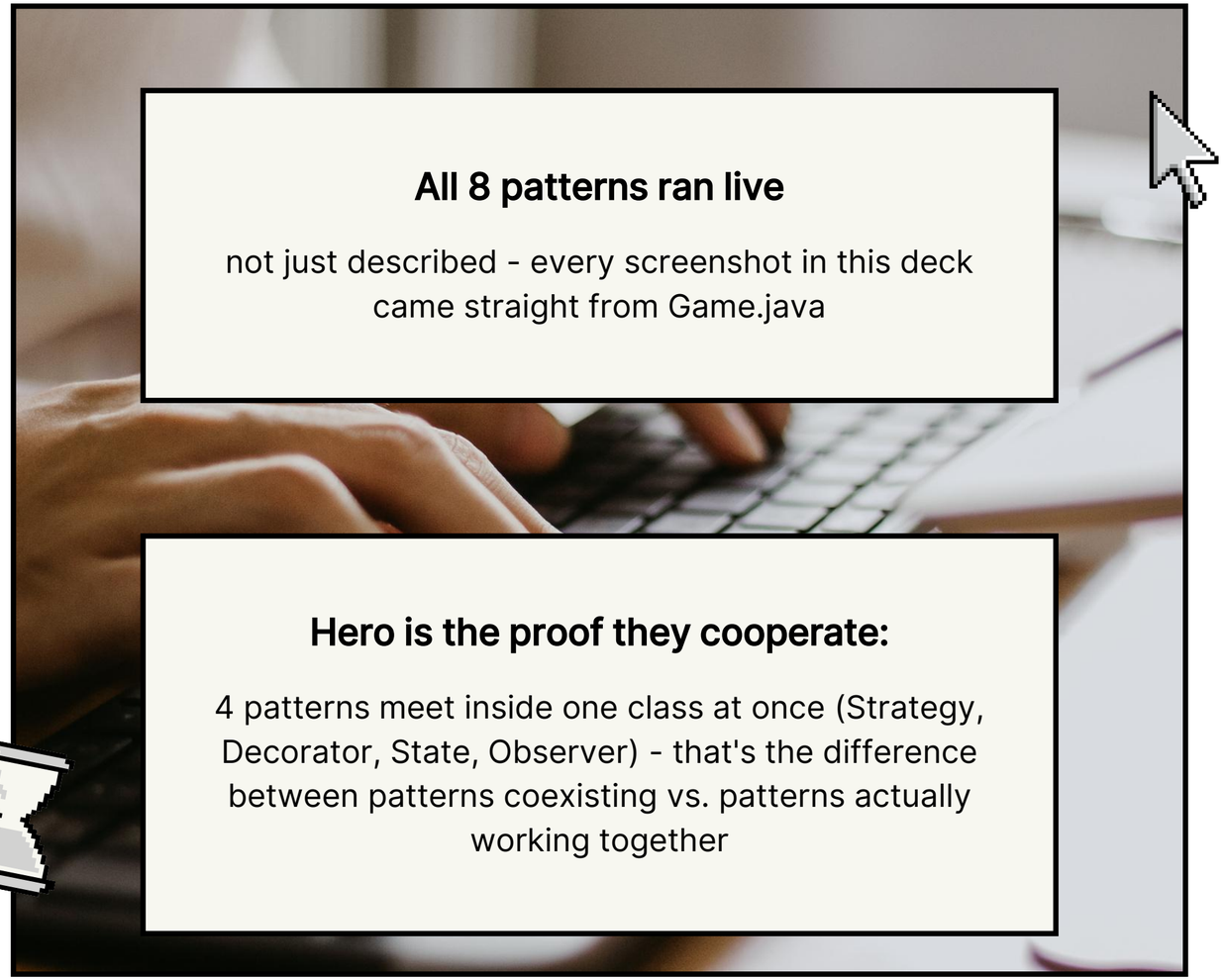
The private constructor blocks anyone from building a HeroHQ directly - getInstance() is the only way in, and it always hands back the same object.

```java
// Singleton - can't be built from outside
private HeroHQ() {  1 usage
}
// The one way to get it:
public static HeroHQ getInstance() {
    return Holder.INSTANCE;
}
```

SUMMARY

Scope stayed narrow on purpose:

console-only, no save system, scripted villains - so the effort went into getting the 8 patterns right, not building a bigger game





THANK YOU

苏睿涵 - L25020028

